deskHPSDR Native RTTY TCI Extension

Protocol documentation - source baseline 2026-08-19 (deskhpsdr-curr 065601)

Scope

This document specifies the deskHPSDR native RTTY extension to TCI. The extension is opt-in per TCI connection and generates ITA2/Baudot RTTY directly in the deskHPSDR TX I/Q path. RTTY receive decoding is outside this extension.

1. Session enable / capability gate

A newly connected TCI client has native RTTY disabled. All other `rtty_*` commands are gated until the client enables the extension.

```
rtty_enable;
RTTY_ENABLE:0;

rtty_enable:1;
RTTY_ENABLE:1;

rtty_enable:0;
RTTY_ENABLE:0;
```

Disabling native RTTY while the client owns an active RTTY transmission performs a hard abort and returns the radio immediately to RX. `rtty_enable` itself does not alter DIGL/DIGU offsets or RTTY parameters.

2. Command summary

Command	Purpose	Query / response
`rtty_enable[:0|1];`	Per-client RTTY opt-in	`RTTY_ENABLE:n;`
`rtty_start;`	Explicitly start native RTTY TX without text	Action; no response
`rtty_text:[...];`	Queue RTTY text; auto-starts TX if idle	Action; no response
`rtty_stop;`	Controlled RTTY stop with MARK tail	Action; no response
`rtty_baud[:v];`	Baud rate	`RTTY_BAUD:v;`
`rtty_shift[:Hz];`	MARK/SPACE shift	`RTTY_SHIFT:n;`
`rtty_reverse[:0|1];`	Swap logical MARK and SPACE	`RTTY_REVERSE:n;`
`rtty_stop_bits[:v];`	Stop bits: 1.0 / 1.5 / 2.0	`RTTY_STOP_BITS:v;`
`rtty_uos[:0|1];`	Unshift On Space	`RTTY_UOS:n;`
`rtty_fill[:MARK|SPACE|LTRS];`	Idle fill mode	`RTTY_FILL:name;`
`rtty_start_crlf[:0|1];`	Initial CR/LF after prefix	`RTTY_START_CRLF:n;`
`rtty_idle_timeout[:s];`	Fill idle timeout, 1..300 s	`RTTY_IDLE_TIMEOUT:n;`

3. Defaults and ranges

Parameter	Default	Valid values / range
Baud	45.4545454545	10.0 ... 300.0
Shift	170 Hz	1 ... 2000 Hz
Reverse	0	0 / 1
Stop bits	1.5	1.0 / 1.5 / 2.0

Parameter	Default	Valid values / range
UOS	1	0 / 1
Fill	LTRS	MARK / SPACE / LTRS
Start CR/LF	1	0 / 1
Idle timeout	25 s	1 ... 300 s

Baud values such as `45.45` are valid and are not forced to the default.

4. Explicit start and automatic start

```
rtty_start;
```

`rtty_start;` starts an RTTY session without user text. It performs the same DIGL/DIGU, transmitter-busy and ownership checks as `rtty_text`. If the same owner already has an active RTTY session, the command is idempotent: it does not restart the prefix and does not clear queued text.

A newly started session emits:

• 300 ms logical MARK prefix.

• Three ITA2 LTRS characters.

• Optional CR + LF when RTTY_START_CRLF is enabled.

• Then the configured fill while waiting for text.

`rtty_text:[...];` retains automatic start behavior, so clients do not have to send `rtty_start;` before every transmission.

5. Sending text

```
rtty_text:[CQ CQ DE DL1BZ\r\nTEST: 12345; NR: 001\r\nTU];
```

The bracketed payload is mandatory. Inside `[...]`, colon, semicolon and comma are payload characters. Only the semicolon following the closing bracket terminates the TCI command.

Supported transport escapes:

```
\r CR\n\n LF\n\\ backslash
```

An actual CR/LF pair is normalized to one ITA2 CR/LF pair. A single LF is encoded as CR + LF. An empty `rtty_text:[];` is ignored and does not start TX. Unsupported characters not present in the deskHPSDR ITA2 table are encoded as SPACE.

6. Fill modes and idle timeout

```
rtty_fill:MARK;
rtty_fill:SPACE;
rtty_fill:LTRS;
```

• **MARK** - continuous logical MARK carrier.

• **SPACE** - continuous logical SPACE carrier.

• **LTRS** - repeated ITA2 LTRS characters including normal Baudot framing.

RTTY_REVERSE applies uniformly to data, prefix, MARK fill, SPACE fill and tail. New text immediately leaves continuous MARK/SPACE fill and resumes the data stream. The idle timer runs while the queue is empty for all three fill modes.

When the idle timeout expires, deskHPSDR sends a 300 ms logical MARK tail and returns to RX.

7. Stop and failure behavior

```
rtty_stop;
```

`rtty_stop;` is the controlled stop: queued/fill operation ends, a 300 ms logical MARK tail is transmitted, then RX is entered.

The following conditions instead perform a **hard abort** and force RX immediately, without waiting for the idle timeout or controlled tail:

• The owning TCI client disconnects.

• The owning client sends rtty_enable:0.

• The owning client is stopped through the generic TCI stop handling.

• The radio mode changes from DIGL/DIGU to a non-DIG mode.

8. TX ownership

Native RTTY has a dedicated TCI RTTY owner. The first enabled client that successfully starts native RTTY becomes the owner. While active, another client cannot start native RTTY or modify RTTY parameters. Stale ownership is cleared when the engine is no longer active. A client also cannot start native RTTY when another TCI TX owner or another transmission already occupies the transmitter.

9. Supported modes and DIGI offsets

Native RTTY TX is accepted only in DIGL or DIGU. Changing DIGL <-> DIGU while active is supported. Changing to any other mode aborts RTTY and forces RX.

Native RTTY does not contain fixed 2210 Hz or 1500 Hz corrections. The I/Q generator reads the current deskHPSDR DIGI offsets at run time:

```
DIGL -> current digi_offset_l
DIGU -> current digi_offset_u
```

Existing TCI commands remain the mechanism for querying or changing those offsets:

```
digl_offset;
digu_offset;

digl_offset:2210;
digu_offset:1500;
```

The values 2210 Hz and 1500 Hz are client/application choices, not constants in the native RTTY engine.

10. RF / Reverse convention

With RTTY_REVERSE=0, logical MARK and SPACE are mapped by the native I/Q path using the current DIGI offset and configured shift. RTTY_REVERSE=1 swaps logical MARK and SPACE uniformly; it is independent of DIGL/DIGU selection.

11. Transport limits and WebSocket framing

• Maximum complete TCI text WebSocket message: 8192 bytes.

• Fragmented WebSocket text messages are reassembled before TCI parsing.

• If the accumulated message exceeds 8192 bytes, the entire remaining WebSocket message is discarded.

• Per-client reassembly state is released on disconnect.

• Native ITA2 symbol queue size: 32768 symbols.

• Character enqueueing is atomic with respect to required FIGS/LTRS shifts.

12. Native modulation implementation

```
phase-continuous NCO
5 ms linear MARK/SPACE frequency shaping
sample-accurate Baudot timing
32768-symbol ITA2 queue
```

The TX analyzer is fed from the same native RTTY I/Q path used for transmission.

13. Recommended client initialization

```
rtty_enable;
rtty_enable:1;

digl_offset;
digu_offset;

rtty_baud;
rtty_shift;
rtty_reverse;
rtty_stop_bits;
rtty_uos;
rtty_fill;
rtty_start_crlf;
rtty_idle_timeout;
```

Typical 45.45-baud configuration:

```
rtty_baud:45.4545;
rtty_shift:170;
rtty_reverse:0;
rtty_stop_bits:1.5;
rtty_uos:1;
rtty_fill:LTRS;
rtty_start_crlf:1;
rtty_idle_timeout:25;
```

Explicit-start example:

```
rtty_start;
rtty_text:[CQ CQ DE DL1BZ\r\n599 001\r\nTU];
rtty_stop;
```

Auto-start example:

```
rtty_text:[CQ CQ DE DL1BZ\r\n599 001\r\nTU];
```